

FTEC5660 homework2 part1 report

1. system architecture and design decisions

The system architecture is designed with a phased processing and intelligent scoring approach. First, the AI agent extracts key data from the CV and performs internal consistency checks to identify potential issues such as timeline conflicts, contradictory job descriptions, or discrepancies between skills and experience. This initial check accounts for 50% of the total score. Next, the system uses information from the CV to search for potential CV ID lists on LinkedIn and Facebook, primarily through `search_facebook_users` and `search_linkedin_people` tools. Subsequently, the LLM utilizes these ID lists and `get_facebook_profile` and `get_linkedin_profile` tools to retrieve detailed social media profiles. The LLM then scores the retrieved Facebook information (contributing 0.2 to the total score), evaluating its consistency with the CV data. Simultaneously, LinkedIn information is scored (contributing 0.3 to the total score), with a focus on professionalism, work experience match, and skill alignment compared to the CV content. Finally, the system selects the highest-scoring match and calculates a weighted total score, comprising the internal CV consistency check score, Facebook information score, and LinkedIn information score. This comprehensive score serves as a quantitative indicator of CV credibility and is used to generate a detailed verification report.

2. Agent Workflow and tool usage strategy

The Agent system employs a fixed workflow to efficiently complete the CV verification task. This process coordinates custom functions with MCP server tools to automate information extraction, validation, and scoring. First, the system uses the `information_extraction` function to process the input PDF resume, driving the LLM to convert PDF text content into structured JSON format, extracting key information such as name, educational background, work experience, skills, and city, and outputting it in a formatted manner, serving as the basis for subsequent steps. Subsequently, the system calls the `first_rating` function, where the LLM identifies internal inconsistencies in the JSON-formatted resume information, checking for conflicts such as timeline clashes, discrepancies in job descriptions, or mismatches between skills and experience, and generating a preliminary score based on the degree of conflict to assess the resume's inherent reliability. For the Facebook platform, the system uses the `seek_for_Facebook_id` function, which first uses the name information extracted in the first step to search for potential Facebook user ID lists via the MCP tool `search_facebook_users`, and then compares them with the city information in the resume for filtering to return an accurate ID list. After obtaining the IDs, the system calls the MCP tool `get_facebook_profile` to retrieve detailed Facebook personal profiles for these IDs, and then the LLM meticulously compares these profiles with the basic resume information obtained in the first step, verifying education, workplaces, residences, etc., and scoring the authenticity.

and consistency of the Facebook information based on the comparison results. For the LinkedIn platform, the system adopts a similar strategy, with the `seek_for_linkedin_id` function responsible for searching. This function first searches for potential LinkedIn user ID lists on LinkedIn based on the city and name information in the resume, using the MCP tool `search_linkedin_people`, and then uses the industry information in the resume to filter the IDs to find the most relevant LinkedIn profiles. After obtaining the IDs, the system calls the MCP tool `get_linkedin_profile` to retrieve detailed LinkedIn professional profiles for these IDs, and the LLM will then perform an in-depth comparison and verification of the retrieved LinkedIn data (including education, work experience, skills, etc.) with the resume information, and score the accuracy and consistency of the LinkedIn information based on the comparison results. After completing the internal CV consistency scoring, Facebook verification scoring, and LinkedIn verification scoring, the system aggregates these three sets of scores using a weighted calculation to obtain a final total score, which serves as a comprehensive indicator of the CV's credibility. Finally, the system generates a detailed verification report, clearly presenting the scores from each stage, any inconsistencies found, and the final comprehensive assessment.

3. Sample verification results

CV1

```
▶ all_score[0] = await generate_score_all_agent(all_cvs[0]["text"])

... =====basic_info=====
{'name': 'John Smith', 'headline': 'Marketing Professional', 'city': ['Singapore', 'Kowloon'], 'country': 'Singapore'}
=====first_rate=====
discrepancy: Significant disconnect between work experience and listed skills.
score_accumulate: 0.38
=====Facebook_rate=====
discrepancy: Different cities found. Conflicting work experience details.
score_accumulate: 0.44
=====LinkedIn_rate=====
discrepancy: Different cities found. Conflicting work experience details. Inconsistent educational dates.
score_accumulate: 0.52
```

CV2

```
▶ all_score[1] = await generate_score_all_agent(all_cvs[1]["text"])

... =====basic_info=====
{'name': 'Minh Pham', 'headline': 'Design Professional', 'city': ['Beijing', 'Hong Kong'], 'country': 'China', 'industry': 'Technology'}
=====first_rate=====
discrepancy: No inconsistencies found.
score_accumulate: 0.50
=====Facebook_rate=====
discrepancy: Inconsistent educational institutions or degrees found. Conflicting work experience details.
score_accumulate: 0.57
=====LinkedIn_rate=====
discrepancy: Different cities found. Inconsistent educational institution or graduation dates.
score_accumulate: 0.72
```

CV3

```
▶ all_score[2] = await generate_score_all_agent(all_cvs[2]["text"])

... =====basic_info=====
{'name': 'Wei Zhang', 'headline': 'Consulting Professional', 'city': ['Munich', 'Sydney'], 'country': 'Germany',
=====first_rate=====
discrepancy: Starting work before latest education graduation.
socre_accumulate: 0.38
=====Facebook_rate=====
discrepancy: Inconsistent educational institutions.
socre_accumulate: 0.51
=====Linkedin_rate=====
discrepancy: Different cities found. Inconsistent educational institutions. Conflicting work experience details.
socre_accumulate: 0.58
```

CV4

```
▶ all_score[3] = await generate_score_all_agent(all_cvs[3]["text"])

... =====basic_info=====
{'name': 'Rahul Sharma', 'headline': 'Legal Professional', 'city': ['Singapore', 'Philippines'], 'country': None,
=====first_rate=====
discrepancy: Overlapping work experience dates, starting work before latest education completion, a significant d
socre_accumulate: 0.00
=====Facebook_rate=====
discrepancy: Different cities found. Inconsistent educational institutions or degrees. Conflicting work experienc
socre_accumulate: 0.00
=====Linkedin_rate=====
discrepancy: Different cities found. Conflicting work experience details. Inconsistent educational institutions o
socre_accumulate: 0.07
```

CV5

```
分 ▶ all_score[4] = await generate_score_all_agent(all_cvs[4]["text"])

... =====basic_info=====
{'name': 'Rahul Sharma', 'headline': 'AI Professional', 'city': ['London', 'Hong Kong', 'Singapore'], 'country':
=====first_rate=====
discrepancy: Found overlapping work experience dates and starting work before graduation.
socre_accumulate: 0.25
=====Facebook_rate=====
discrepancy: Conflicting work experience details.
socre_accumulate: 0.38
=====Linkedin_rate=====
discrepancy: Different cities found. Inconsistent educational institutions or graduation dates. Conflicting work
socre_accumulate: 0.46
```

Evaluation result:

```
scores = all_score # Your code should generate this list [0.2, 0.3, 0.4, 0.5, 0.6]
groundtruth = [1, 1, 1, 0, 0] # Do not modify
```

```
result = evaluate(scores, groundtruth)
print(result)
```

```
{'decisions': [1, 1, 1, 0, 0], 'correct': 5, 'total': 5, 'final_score': 1.0}
```